

Typing Race Simulator Report

Encapsulation in the Typist Class

In the Typist class I made the main variables private, such as the typist's name, symbol, progress, accuracy and burnout information. I did this because these values shouldn't be changeable directly from outside the class. Instead I have used methods to get or change the values properly. For example, `getProgress()`, `getAccuracy()`, `getName()` and `isBurntOut()` let other parts of the program check the typist's details without directly accessing the variables. For accuracy, I used `setAccuracy()` so the value stays between 0.0 and 1.0. If someone tries to set it too low, it becomes 0.0, and if it is too high, it becomes 1.0. This stops invalid accuracy values being used. I also made sure `slideBack()` does not let the typist's progress go below zero. This is needed because otherwise a typist could end up before the start of the race, which would not make sense.

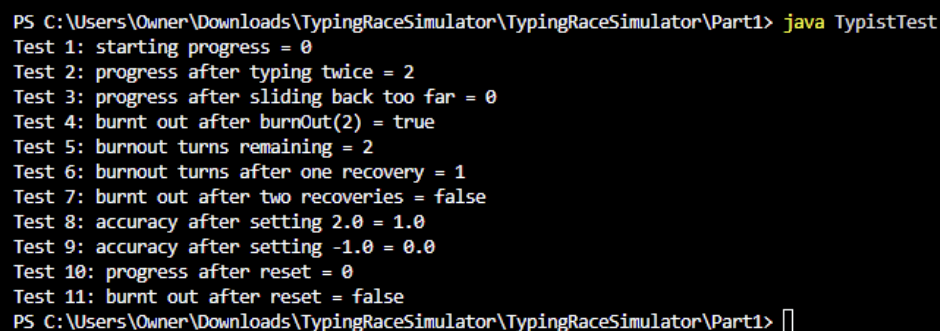
Testing the Typist Class

I tested the Typist class using a temporary test file called `TypistTest.java`. I used this just to check the main behaviours before deleting it from the final submission.

The tests check that:

- progress starts at zero
- `typeCharacter()` increases progress
- `slideBack()` does not let progress go below zero
- `burnOut()` puts the typist into a burnt-out state
- `recoverFromBurnout()` reduces the burnout counter
- burnout stops when the counter reaches zero
- `setAccuracy()` keeps accuracy between 0.0 and 1.0
- `resetToStart()` resets progress and burnout

Figure 1: TypistTest output showing Typist class behaviour.



```
PS C:\Users\Owner\Downloads\TypingRaceSimulator\TypingRaceSimulator\Part1> java TypistTest
Test 1: starting progress = 0
Test 2: progress after typing twice = 2
Test 3: progress after sliding back too far = 0
Test 4: burnt out after burnOut(2) = true
Test 5: burnout turns remaining = 2
Test 6: burnout turns after one recovery = 1
Test 7: burnt out after two recoveries = false
Test 8: accuracy after setting 2.0 = 1.0
Test 9: accuracy after setting -1.0 = 0.0
Test 10: progress after reset = 0
Test 11: burnt out after reset = false
PS C:\Users\Owner\Downloads\TypingRaceSimulator\TypingRaceSimulator\Part1> 
```

The output shows that the main parts of the Typist class worked correctly, especially progress, slide-back, burnout, accuracy limits and reset behaviour.

Bugs Found in the TypingRace Class

The first issue was that only seat1Typist and seat2Typist were reset at the start of the race, but the race has three seats, so seat3Typist should also be reset. If it is not reset, the third typist may keep old progress or burnout information if the same race object is reused. The second issue was that the program could crash if a seat was empty. This is because the original code sometimes called methods on a typist without checking if the typist was null first. Another issue was the race finish condition - a typist should win when their progress is greater than or equal to the passage length, if the code only checks for the exact passage length, the race may not finish properly if a typist overshoots the finish. I had also found a display issue, if a typist's progress went past the passage length, the race display could show the typist beyond the end of the track. This makes the output look wrong even though the race has already been completed.

Fixes Made to the TypingRace Class

I fixed the reset issue by making the race reset all three typists before the race begins. I also added null checks before calling resetToStart(), so the program only resets a typist if that seat actually contains a typist. For the empty seat issue, I added null checks in the race logic meaning empty seats are skipped safely instead of causing a nullpointerexception. I fixed the race finish condition by using a greater than or equal to check to the passage length. This means a typist is declared the winner as soon as their progress reaches or passes the end of the passage. The display issue was fixed by capping the displayed progress at the passage length which stops the typist symbol from being printed past the finish line. I also kept the existing constants such as MISTYPE_BASE_CHANCE, SLIDE_BACK_AMOUNT and BURNOUT_DURATION as they make the code easier to follow, because the important values have clear understandable names instead of being written as unexplained numbers in the middle of the code.

Figure 2: Text-based race finishing with a winner.

```
[zz] = burnt out    [<] = just mistyped

TYPING RACE ? passage length: 40 chars
=====
|                                     1| TURBOFINGERS (Accuracy: 0.85)
|                                     | QWERTY_QUEEN (Accuracy: 0.6)
|           2                         | HUNT_N_PECK (Accuracy: 0.3)
| 3                                     |
=====
[zz] = burnt out    [<] = just mistyped

And the winner is... TURBOFINGERS!
Final accuracy: 0.85
```

GUI Testing Evidence

For Part 2, I made a GUI version using Java Swing. The GUI has options for the passage length, the number of typists, the typist names, symbols, typing styles, keyboard types, colours and accessories. I also added checkboxes for modifiers such as autocorrect, caffeine mode and night shift. To test it, I messed around with the settings before starting the race. I tested it with six typists and used different names, symbols, styles, keyboards, colours and accessories. This helped me ensure that the GUI was actually using the user's selected choices. When the race finished, the GUI displayed the winner as well as the results. It displayed WPM, accuracy, mistypes, burnouts, best WPM, leaderboard points and a comparison between each of the typists.

Figure 3: GUI setup and results using six typists and custom settings.

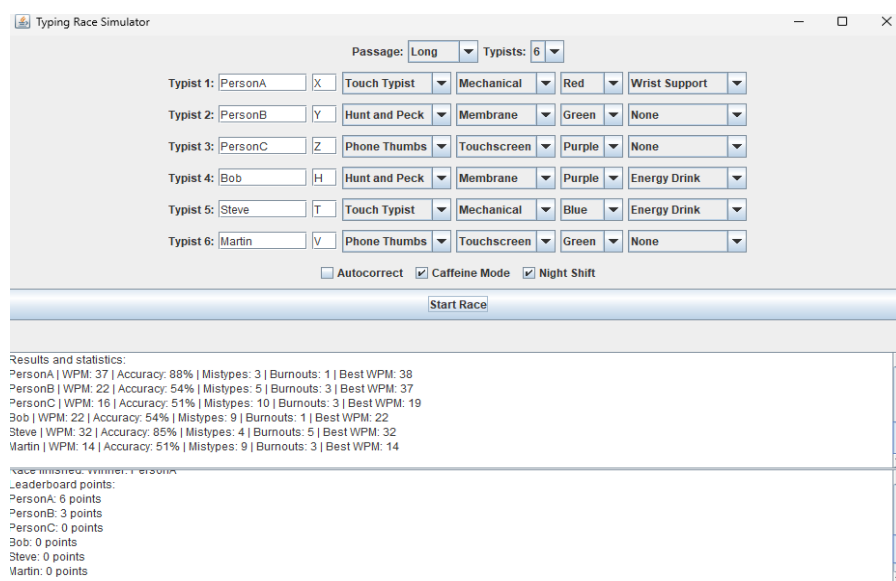
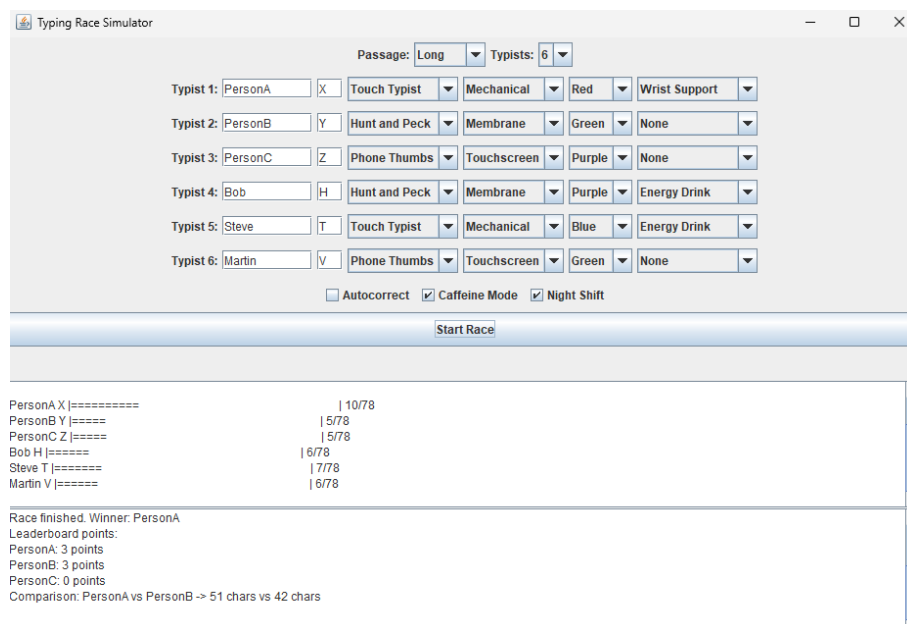


Figure 4: GUI race progress and final output.



Conclusion

Overall, the text-based race and the GUI version both worked after testing. The Typist class stores the information for each typist and the TypingRace class handles the race logic. The main things I fixed were the missing reset for the third typist, the null checks for empty seats, the race finish condition and the display going past the end of the track. The testing showed that progress, slide-back, burnout, accuracy limits and resetting all worked correctly as they should. The GUI also worked with different settings and displayed the race in real time, as well as the final results clearly.